

EXPERIMENT: Feed forward neural Network

Aim

To implement a Feed Forward Neural Network (FFNN) in Python for classification using forward propagation.

Objectives

1. To understand the structure of a Feed Forward Neural Network.
2. To implement input, hidden and output layers.
3. To understand forward propagation.
4. To use an activation function for classification.
5. To train the network and generate predictions.

Algorithm

1. Initialize the input data.
2. Initialize weights and biases randomly.
3. Pass the input data to the hidden layer.
4. Calculate the weighted sum.
5. Apply the sigmoid activation function.
6. Pass the hidden-layer output to the output layer.
7. Calculate the output weighted sum.
8. Apply the sigmoid function.
9. Calculate the error.
10. Update weights using backpropagation.
11. Repeat the process for several epochs.
12. Generate the final predictions.

Code

```
import random
```

```
import math
```

```
X = [  
    [0, 0],  
    [0, 1],  
    [1, 0],  
    [1, 1]
```

```

]
Y = [
    [0],
    [1],
    [1],
    [0]
]

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

input_neurons = 2
hidden_neurons = 4
output_neurons = 1
learning_rate = 0.5
epochs = 10000

W1 = [
    [random.uniform(-1, 1) for j in range(hidden_neurons)]
    for i in range(input_neurons)
]

W2 = [
    [random.uniform(-1, 1) for j in range(output_neurons)]
    for i in range(hidden_neurons)
]

B1 = [0.0 for i in range(hidden_neurons)]
B2 = [0.0 for i in range(output_neurons)]

for epoch in range(epochs):
    for sample in range(len(X)):
        hidden_input = []
        for j in range(hidden_neurons):
            total = B1[j]

```

```

    for i in range(input_neurons):
        total += X[sample][i] * W1[i][j]
    hidden_input.append(total)
hidden_output = [
    sigmoid(value) for value in hidden_input
]
output_input = []
for j in range(output_neurons):
    total = B2[j]
    for i in range(hidden_neurons):
        total += hidden_output[i] * W2[i][j]
    output_input.append(total)

output = [
    sigmoid(value) for value in output_input
]
error = [
    Y[sample][j] - output[j]
    for j in range(output_neurons)
]
output_delta = [
    error[j] * sigmoid_derivative(output[j])
    for j in range(output_neurons)
]
hidden_delta = []
for i in range(hidden_neurons):
    total = 0
    for j in range(output_neurons):
        total += output_delta[j] * W2[i][j]
    delta = total * sigmoid_derivative(hidden_output[i])
    hidden_delta.append(delta)

```

```

for i in range(hidden_neurons):
    for j in range(output_neurons):
        W2[i][j] += (
            learning_rate
            * hidden_output[i]
            * output_delta[j]
        )
    for j in range(output_neurons):
        B2[j] += learning_rate * output_delta[j]
    for i in range(input_neurons):
        for j in range(hidden_neurons):
            W1[i][j] += (
                learning_rate
                * X[sample][i]
                * hidden_delta[j]
            )
        for j in range(hidden_neurons):
            print("=====")
            print(" FEED FORWARD NEURAL NETWORK")
            print(" XOR CLASSIFICATION")
            print("=====")
            print("\nPredictions:")
            print("-----")
            correct = 0
            hidden_output = []
            for j in range(hidden_neurons):
                total = B1[j]
                for i in range(input_neurons):
                    total += X[sample][i] * W1[i][j]
                hidden_output.append(sigmoid(total))
            total = B2[0]
            for i in range(hidden_neurons):
                total += hidden_output[i] * W2[i][0]

```

```
prediction = sigmoid(total)
predicted_class = 1 if prediction >= 0.5 else 0
if predicted_class == Y[sample][0]:
    correct += 1
print(
    X[sample],
    "->",
    round(prediction, 3),
    "Class:",
    predicted_class
)
accuracy = (correct / len(X)) * 100
print("-----")
print("Accuracy:", accuracy, "%")
```

Output



```
IDLE Shell 3.12.5
File Edit Shell Debug Options Window Help
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/shaik arshad basha/OneDrive/Desktop/Map Coloring using CSP.py
=====
FEED FORWARD NEURAL NETWORK
XOR CLASSIFICATION
=====

Predictions:
-----
[0, 0] -> 0.009 Class: 0
[0, 1] -> 0.985 Class: 1
[1, 0] -> 0.986 Class: 1
[1, 1] -> 0.022 Class: 0
-----
Accuracy: 100.0 %
>>> |
```

Ln: 18 Col: 0

Result

The Feed Forward Neural Network was successfully implemented using forward propagation and backpropagation. The network learned the XOR classification problem and produced predictions close to the expected outputs.

Conclusion

The experiment demonstrates the working of a Feed Forward Neural Network consisting of an input layer, hidden layer, and output layer. The network learns by adjusting weights using backpropagation and can be applied to classification and prediction tasks.